



UNIVERSITY OF CAPE TOWN

CSC2042S: Machine Learning

Author: George Goodey

September 16, 2026

Contents

1	Supervised learning	2
1.1	What is machine learning?	2
1.2	Types of machine learning	2
1.2.1	Unsupervised learning	2
1.3	Simple linear regression	2
1.3.1	Loss function	3
1.4	Logistic regression	5
1.4.1	Probability basics	5
1.4.2	Discriminative classifier	5
1.4.3	Logistic regression	5
1.4.4	Example	7
1.5	Cross-entropy loss	7
1.5.1	Log probabilities	7
1.5.2	Loss function	7
1.5.3	Example	8
1.5.4	Objective function	8
1.5.5	Optimization	9
1.6	Gradient descent	9
1.6.1	Loss function	9
1.6.2	Optimization	9
1.6.3	Learning rate	10
1.6.4	Optimization (continued)	11
1.6.5	Gradient descent formula	11
1.7	Stochastic gradient descent	11
1.7.1	Batching	11
1.7.2	Example	12
1.7.3	Vectorization	12

1 Supervised learning

1.1 What is machine learning?

Definition: Machine Learning

Machine learning is the study of algorithms that:

- improve their performance P (according to a criterion that measures success)
- at some task T
- with experience E (from data examples).

A well-defined task is given by $\langle P, T, E \rangle$. For example:

- **Image classification:** $\langle \text{Accuracy, Classify handwritten digits, Dataset of labelled images} \rangle$.
- **Chess:** $\langle \text{Playing quality, Play chess against human, Dataset of old games} \rangle$.
- **ChatGPT:** $\langle \text{Quality of conversation, Generate responses, All text on the web} \rangle$.

1.2 Types of machine learning

There are many types:

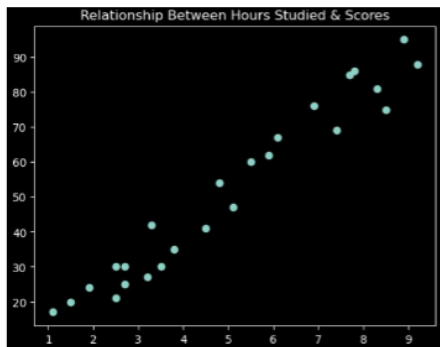
- Supervised learning.
- Unsupervised learning.
- Reinforcement learning.
- etc.

1.2.1 Unsupervised learning

The problem is framed as finding structure in x without labels (no y). Inputs are examples in the dataset represented by *features*. Data D is $|D|$ examples $\{x_1, x_2, \dots, x_{|D|}\}$ without any labels. Given a new example x , where does this fit into the data?

1.3 Simple linear regression

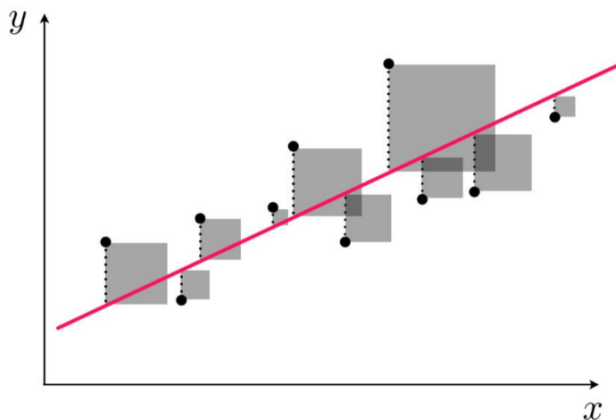
A simple linear regression model predicts the output as a linear function of one input feature x : $y = wx + b$, otherwise known as a straight line function. Changes in x imply a proportional change in y . Even if the relationship is not perfectly linear, linear regression can still be effective as an approximation or baseline model.



1.3.1 Loss function

For each input, linear regression produces an estimate $\hat{y}_i = w \cdot x_i + b$. We want to set w and b to minimize the distance between our estimate $\hat{y}_i$ (prediction) and the true y_i . We want to minimize this distance/error across our entire training set $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$.

A principled way to define the learning problem is with a loss function (also known as a cost function): $L(\hat{y}_i, y_i)$ measuring how close the model prediction $\hat{y}_i$ is to the observed data y_i . If the model fits the data well, so that $y_i \approx \hat{y}_i$, then the loss function should take a small value, and vice versa. For this, we can use the squared error loss.



The squared error loss measures how good your linear regression model with parameters w, b fits the data. For a single data point (x_i, y_i) :

$$\begin{aligned} L(\hat{y}_i, y_i) &= (y_i - \hat{y}_i)^2 \\ &= (y_i - f(x_i; w, b))^2 \\ &= (y_i - (w \cdot x + b))^2 \\ &= (y_i - w \cdot x - b)^2 \end{aligned}$$

We want to find the weight that minimizes the loss, averaged over all examples in our training set.

$$J(w, b) = \frac{1}{N} \sum_{i=1}^N L(\hat{y}_i, y_i)$$

Note: Why as a function of w and b ?

These are the variables we want to learn!

All examples (x_i, y_i) in the function are determined by whatever is in the training set. This is out of our control. Now, we want to find:

$$\hat{w}, \hat{b} = \underset{w, b}{\operatorname{argmin}} J(w, b)$$

For linear regression, the loss function (mean squared error) is convex. This means it has a single global minimum and no local minima. Linear regression has a closed form solution. The question is, how do we find $\hat{w}, \hat{b} = \underset{w, b}{\operatorname{argmin}} J(w, b)$?

1. Take the partial derivative of J with respect to w and b .
2. Set the partial derivative equal to 0.
3. Solve for w and b .

Derivation

Derivation w.r.t b :

$$\begin{aligned}\frac{\partial}{\partial b} J(w, b) &= \frac{\partial}{\partial b} \frac{1}{N} \sum_{i=1}^N (y_i - wx_i - b)^2 \\ &= \frac{-2}{N} \sum_{i=1}^N (y_i - wx_i - b) \\ 0 &= \sum_{i=1}^N (y_i - wx_i - b) \quad (\text{Setting equal to 0}) \\ 0 &= \sum_{i=1}^N y_i - w \sum_{i=1}^N x_i - Nb \\ \hat{b} &= \bar{y} - \hat{w}\bar{x}\end{aligned}$$

Derivation w.r.t w :

Before differentiating w.r.t w , substitute this expression for b back into the cost so that J becomes a function of w only:

$$J(w) = \frac{1}{N} \sum_{i=1}^N \left[y_i - wx_i - (\bar{y} - w\bar{x}) \right]^2 = \frac{1}{N} \sum_{i=1}^N \left[(y_i - \bar{y}) - w(x_i - \bar{x}) \right]^2$$

Now differentiate w.r.t w :

$$\frac{d}{dw} J(w) = \frac{-2}{N} \sum_{i=1}^N \left[(y_i - \bar{y}) - w(x_i - \bar{x}) \right] (x_i - \bar{x})$$

Set the derivative to 0 and simplify:

$$0 = \frac{-2}{N} \left[\sum_{i=1}^N (y_i - \bar{y})(x_i - \bar{x}) - w \sum_{i=1}^N (x_i - \bar{x})^2 \right]$$

$$w \sum_{i=1}^N (x_i - \bar{x})^2 = \sum_{i=1}^N (y_i - \bar{y})(x_i - \bar{x})$$

$$\hat{w} = \frac{\sum_{i=1}^N (y_i - \bar{y})(x_i - \bar{x})}{\sum_{i=1}^N (x_i - \bar{x})^2}$$

1.4 Logistic regression

1.4.1 Probability basics

- $P(A)$: probability of event A .
- $P(A^c)$: probability of event A not happening.
- $P(A|B)$: probability of A given B (conditional probability).
- For any event A , $0 \leq P(A) \leq 1$.
- Let S be the sample space of all possible events, then $P(S) = 1$.
- For any event A , $P(A^c) = 1 - P(A) \iff P(A) = 1 - P(A^c)$.

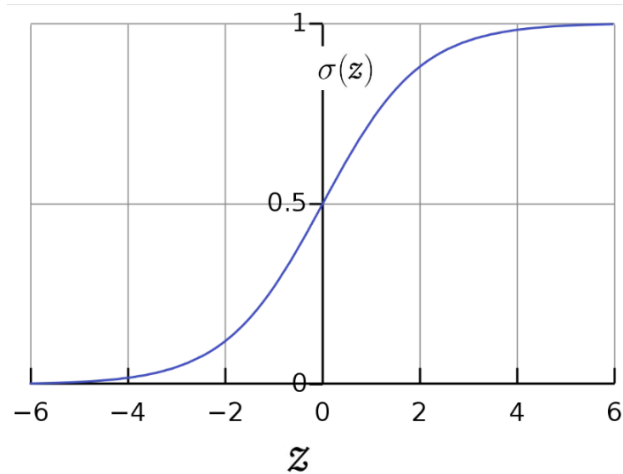
1.4.2 Discriminative classifier

Discriminative models learn boundaries between classes. For example, suppose we are classifying cat vs. dog images. The input x is the image (matrix of pixel values). The output y is the class (either cat or dog). The goal of classification is to find the correct class y given input x . To classify, we compute: $y = \underset{y \in C}{\operatorname{argmax}} P(y|x)$.

1.4.3 Logistic regression

We introduce the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



The sigmoid function is called a logistic function. It squeezes any input value between 0 and 1. To classify the input $\mathbf{x}$:

- Compute the weighted sum, $z = \sum w_i x_i + b = \mathbf{w} \cdot \mathbf{x} + b$.
- Pass z through the sigmoid function, $\sigma(\mathbf{w} \cdot \mathbf{x} + b)$.
- Treat the result as a probability.

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x} - b)}$$

A key property of the sigmoid function: $1 - \sigma(z) = \sigma(-z)$.

Derivation

$$\begin{aligned} P(y = 0|\mathbf{x}) &= 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\ &= 1 - \frac{1}{1 + \exp(-\mathbf{w} \cdot \mathbf{x} - b)} \\ &= \frac{\exp(-\mathbf{w} \cdot \mathbf{x} - b)}{1 + \exp(-\mathbf{w} \cdot \mathbf{x} - b)} \times \frac{\exp(\mathbf{w} \cdot \mathbf{x} + b)}{\exp(\mathbf{w} \cdot \mathbf{x} + b)} \\ &= \frac{1}{1 + \exp(\mathbf{w} \cdot \mathbf{x} + b)} \\ &= \sigma(-\mathbf{w} \cdot \mathbf{x} - b) \end{aligned}$$

Now, we have a way to compute both $P(y = 1|\mathbf{x})$ and $P(y = 0|\mathbf{x})$. We can now make decisions:

$$\text{decision}(\mathbf{x}) = \begin{cases} 1 & \text{if } P(y = 1|\mathbf{x}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

The decision boundary of $P(y|x) = 0.5$ is defined where $\mathbf{w} \cdot \mathbf{x} + b = 0$. This is the default decision boundary in logistic regression. However, you can specify a different decision boundary to suit your task.

- Choose a higher value, like $P(y = 1|\mathbf{x}) > 0.9$, if you want the model to be confident before flagging an instance of a positive class.
- Choose a lower value, like $P(y = 1|\mathbf{x}) > 0.2$, if you don't want to risk missing instances of the positive class (e.g. medical diagnosis).

1.4.4 Example

Predict if a fire weather warning should be announced.

- x_1 : temperature in C° .
- x_2 : wind speed in km/h.
- $y = 0$: no warning needed.
- $y = 1$: warning needed.

Suppose the weights are $\mathbf{w} = [0.1, 0.05]$ and the bias term is $b = -5$. For an input vector $\mathbf{x} = [25, 60]$,

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \sigma((0.1)(25) + (0.05)(60) - 5) = \sigma(0.5) = 0.622$$

$$P(y = 0|\mathbf{x}) = 1 - P(y = 1|\mathbf{x}) = 1 - 0.622 = 0.378$$

Since $P(y = 1|\mathbf{x}) = 0.622 > 0.5$, a warning should be given.

1.5 Cross-entropy loss

1.5.1 Log probabilities

Definition: Log probability

A log probability is a logarithm of a probability value, $\log P(X)$. This represents probabilities on a logarithmic scale $(-\infty, 0]$ instead of the usual $[0, 1]$ probability interval.

A useful property of log properties:

$$\log P(ABC) = \log(P(A) \times P(B) \times P(C)) = \log P(A) + \log P(B) + \log P(C)$$

1.5.2 Loss function

We use conditional **maximum likelihood estimation**:

- Find parameters that **maximize the probability of the true labels** in the training data given the observations.
- For binary classification, since there are 2 discrete outcomes, we can express the classifier's probability of the true label y as:

$$P(y|\mathbf{x}) = \hat{y}^y (1 - \hat{y})^{1-y}$$

- If $y = 1$, this simplifies to $\hat{y}$ which is $P(y = 1|\mathbf{x})$.
- If $y = 0$, this simplifies to $1 - \hat{y}$ which is $P(y = 0|\mathbf{x})$.

We can rewrite this in terms of log probabilities:

$$\begin{aligned}\log P(y|\mathbf{x}) &= \log [\hat{y}^y (1 - \hat{y}^{1-y})] \\ &= y \log \hat{y} + (1 - y) \log(1 - \hat{y})\end{aligned}$$

We can then turn this into a loss function to be minimized by using the negative log likelihood. This is called the **cross-entropy loss** between the prediction and the true class.

$$\begin{aligned}L_{\text{CE}}(\hat{y}, y) &= -\log P(y|\mathbf{x}) \\ &= -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log[\sigma(\mathbf{w} \cdot \mathbf{x} + b)] + (1 - y) \log[\sigma(-\mathbf{w} \cdot \mathbf{x} - b)]]\end{aligned}$$

By minimizing this loss, we maximize the probabilities assigned to the true labels by the logistic regression model.

1.5.3 Example

Predict if a fire weather warning should be announced.

- x_1 : temperature in C° .
- x_2 : wind speed in km/h.
- $y = 0$: no warning needed.
- $y = 1$: warning needed.

Suppose the weights are $\mathbf{w} = [0.1, 0.05]$ and the bias term is $b = -5$.

For an input vector $\mathbf{x} = [25, 60]$ and true label $y = 1$:

$$\begin{aligned}P(y = 1|\mathbf{x}) &= \sigma(\mathbf{w} \cdot \mathbf{x} + b) = 0.622 \\ L_{\text{CE}}(\hat{y}, y) &= -\log P(y = 1|\mathbf{x}) = -\log 0.622 = 0.206\end{aligned}$$

For an input vector $\mathbf{x} = [25, 60]$ and true label $y = 0$:

$$\begin{aligned}P(y = 0|\mathbf{x}) &= 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) = 0.378 \\ L_{\text{CE}}(\hat{y}, y) &= -\log P(y = 0|\mathbf{x}) = -\log 0.378 = 0.423\end{aligned}$$

Since the loss of true label $y = 1$ is lower, we should give a warning.

Note

The loss is higher if the model is wrong. Lower probabilities assigned to the true class implies a higher loss.

1.5.4 Objective function

Given the cross-entropy loss, we want to minimize this loss on average across our training set:

$$\frac{1}{N} \sum_{i=1}^N L_{\text{CE}}(\hat{y}_i, y_i)$$

If we can adjust parameters $\mathbf{w}$ and b to lower the loss over all instances in the training set, we will have a good classifier. This is the essence of modern machine learning and deep learning.

1.5.5 Optimization

Let the loss function be parameterized by parameters $\theta = (\mathbf{w}, b)$. We represent $\hat{y}$ as $f(\mathbf{x}; \theta)$. We want to find parameters that minimize the loss averaged over all examples in our training set.

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N L_{\text{CE}}(f(\mathbf{x}; \theta), y_i)$$

There is **no closed form solution** for finding minimizing weights for logistic regression. Instead, we will use an iterative method called gradient descent.

1.6 Gradient descent

Definition: Gradient descent

Gradient descent is an optimization algorithm that iteratively updates parameters by moving in the opposite direction of the gradient of the loss function, in order to minimize the loss.

1.6.1 Loss function

We want parameters $\mathbf{w}, b$ that minimize loss averaged over m examples in our training set. We denote the average loss as $J(\mathbf{w}, b)$ with $\hat{y}_i = f(\mathbf{x}_i; \mathbf{w}, b)$ written as a function of inputs and parameters.

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m [y_i \log \sigma(\mathbf{w} \cdot \mathbf{x}_i + b) + (1 - y_i) \log [1 - \sigma(\mathbf{w} \cdot \mathbf{x}_i + b)]]$$

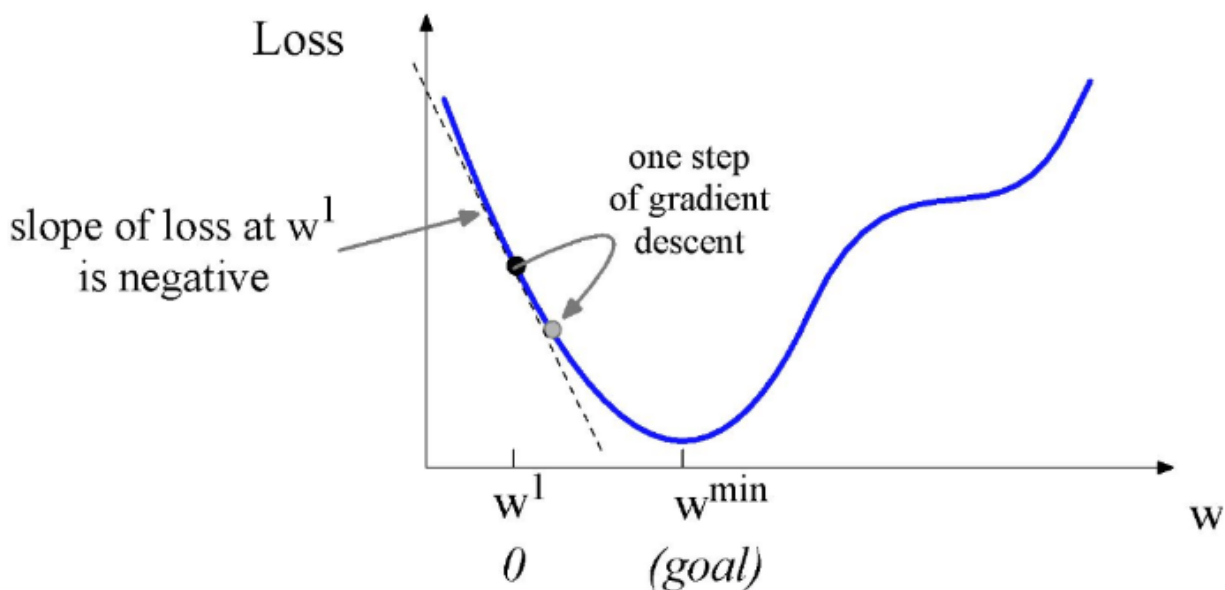
Now, we want to find:

$$\hat{\mathbf{w}}, \hat{b} = \underset{\mathbf{w}, b}{\operatorname{argmin}} J(\mathbf{w}, b)$$

1.6.2 Optimization

For logistic regression, the loss function is convex. Gradient descent from any point is guaranteed to find the minimum. Here is a visualization for a single scalar w : Given the current w , move in the reverse direction from the function slope.

L



The loss is a function of multiple parameters $w_1, \dots, w_n, b$:

$$L_{\text{CE}} = -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1 - y) \log[1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b)]]$$

The gradient of this loss with respect to the parameters θ will be a vector of partial derivatives.

$$\nabla_{\theta} L = \frac{\partial L}{\partial \theta} = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_n} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

For models with n -dimensional weight vectors:

- For each dimension w_i , the gradient component i tells us the slope w.r.t that parameter.
- We express the slope as the partial derivative of L w.r.t w_i .
- The gradient is defined as the vector of these partial derivatives.

1.6.3 Learning rate

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \frac{d}{dw} L(f(\mathbf{x}; \mathbf{w}), y)$$

The gradient tells us:

- **Sign:** which way to move (left, right, up, down).
- **Magnitude:** how steep/big the update is.

To control the step size, the value of the gradient is weighted by a learning rate η . A higher value of η means $\mathbf{w}$ changes faster. This is considered a hyperparameter.

- If the learning rate is too high, it will take big steps and overshoot.
- If the learning rate is too low, it will take too long to converge.

1.6.4 Optimization (continued)

How do we find $\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(\mathbf{x}; \theta), y_i)$? We do the following iteratively:

- Compute the derivative of the loss function with respect to the parameters.
- Update the parameters to move in the opposite direction of the gradient.

The partial derivatives for logistic regression:

$$\boxed{\frac{\partial L}{\partial w_i} = [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y]x_i} \quad \boxed{\frac{\partial L}{\partial b} = \sigma(\mathbf{w} \cdot \mathbf{x} + b) - y}$$

1.6.5 Gradient descent formula

The final question for updating parameters θ based on the gradient is:

$$\theta^{t+1} = \theta^t - \eta \nabla_{\theta} L(f(\mathbf{x}; \theta), y)$$

Note

Gradient descent can be applied to any model with any number of parameters. Gradient descent is a general optimization method, not just for logistic regression!

1.7 Stochastic gradient descent

Definition: Stochastic gradient descent

Stochastic gradient descent (SGD) iteratively updates parameters. It approximates the true gradient by using a subset of data at each step.

At each step, a random subset of examples is sampled. Both the gradient is computed and the parameters are updated based on this subset.

$$\theta^{t+1} = \theta^t - \eta \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i)$$

Batch / mini-batch: The subset of data used for SGD at a particular training step.

Batch size: The number of training examples in each batch. **This is a hyperparameter.**

1.7.1 Batching

In practice, it's computationally expensive to apply gradient descent to the entire training set simultaneously. This is due to the high computational/memory requirements for large datasets and the slow convergence. Instead, we employ the principle of batching.

Traverse the training set in batches, B examples at a time, updating parameters based on gradient w.r.t average loss of the batch.

$$J_B(\theta) = \frac{1}{B} \sum_{i=1}^B L(f(\mathbf{x}_i; \theta), y_i)$$

The gradient of the cross-entropy loss averaged over all B examples in a batch is the average of the individual sample gradients.

$$\nabla_{\theta} J_B(\theta) = \frac{1}{B} \sum_{i=1}^B \nabla_{\theta} L(f(\mathbf{x}_i; \theta), y_i)$$

1.7.2 Example

Assume initial parameters θ^0 are all zero: $w_1 = w_2 = b = 0$. let's perform a SGD update based on one example in the training set. Let $\mathbf{x} = [3, 2]$ with label $y = 1$. Then,

$$\begin{aligned} \nabla_{\theta} &= \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \frac{\partial L}{\partial b} \end{bmatrix} = \begin{bmatrix} [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y]x_1 \\ [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y]x_2 \\ [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y] \end{bmatrix} \\ &= \begin{bmatrix} [\sigma(0) - 1](3) \\ [\sigma(0) - 1](2) \\ [\sigma(0) - 1] \end{bmatrix} \\ &= \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} \end{aligned}$$

Now that we have the gradient vector, we compute the new parameter values with gradient descent. Set the learning rate $\eta = 0.1$.

$$\begin{aligned} \theta^1 &= \theta^0 - \eta \nabla_{\theta} L \\ &= \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - (0.1) \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} \\ &= \begin{bmatrix} -0.15 \\ -0.10 \\ -0.05 \end{bmatrix} \end{aligned}$$

1.7.3 Vectorization

Construct matrices and vectors that allow us to process a batch as a matrix/vector computation. Let the rows of matrix X represent input features of individual trainign samples with B examples in the batch and n input features:

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{B1} & x_{B2} & \cdots & x_{Bn} \end{bmatrix}_{B \times n}$$

The vector $\mathbf{w}$ is the weight parameters. Let $\mathbf{b}$ be a B -dimensional vector repeating bias parameter b .

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}_{n \times 1} \quad \mathbf{b} = \begin{bmatrix} b \\ b \\ \vdots \\ b \end{bmatrix}_{B \times 1}$$

Then we can compute the weighted sums $\mathbf{w} \cdot \mathbf{x}_i + b$ simultaneously for all B examples with matrix multiplication.

$$X\mathbf{w} + \mathbf{b} = \begin{bmatrix} w_1x_{11} + w_2x_{12} + \cdots + w_nx_{1n} + b \\ w_1x_{21} + w_2x_{22} + \cdots + w_nx_{2n} + b \\ \vdots \\ w_1x_{B1} + w_2x_{B2} + \cdots + w_nx_{Bn} + b \end{bmatrix}_{B \times 1}$$

If we apply the sigmoid function to each element of this vector:

$$\hat{\mathbf{y}} = \sigma(X\mathbf{w} + \mathbf{b}) = \begin{bmatrix} \sigma(w_1x_{11} + w_2x_{12} + \cdots + w_nx_{1n} + b) \\ \sigma(w_1x_{21} + w_2x_{22} + \cdots + w_nx_{2n} + b) \\ \vdots \\ \sigma(w_1x_{B1} + w_2x_{B2} + \cdots + w_nx_{Bn} + b) \end{bmatrix}_{B \times 1} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_B \end{bmatrix}$$